



LẬP TRÌNH GAME CƠ BẢN

Giảng viên: Ths. Nguyễn Tuấn Minh

Buổi 2 : Kiến trúc và kỹ thuật lập trình game

Mục tiêu buổi học

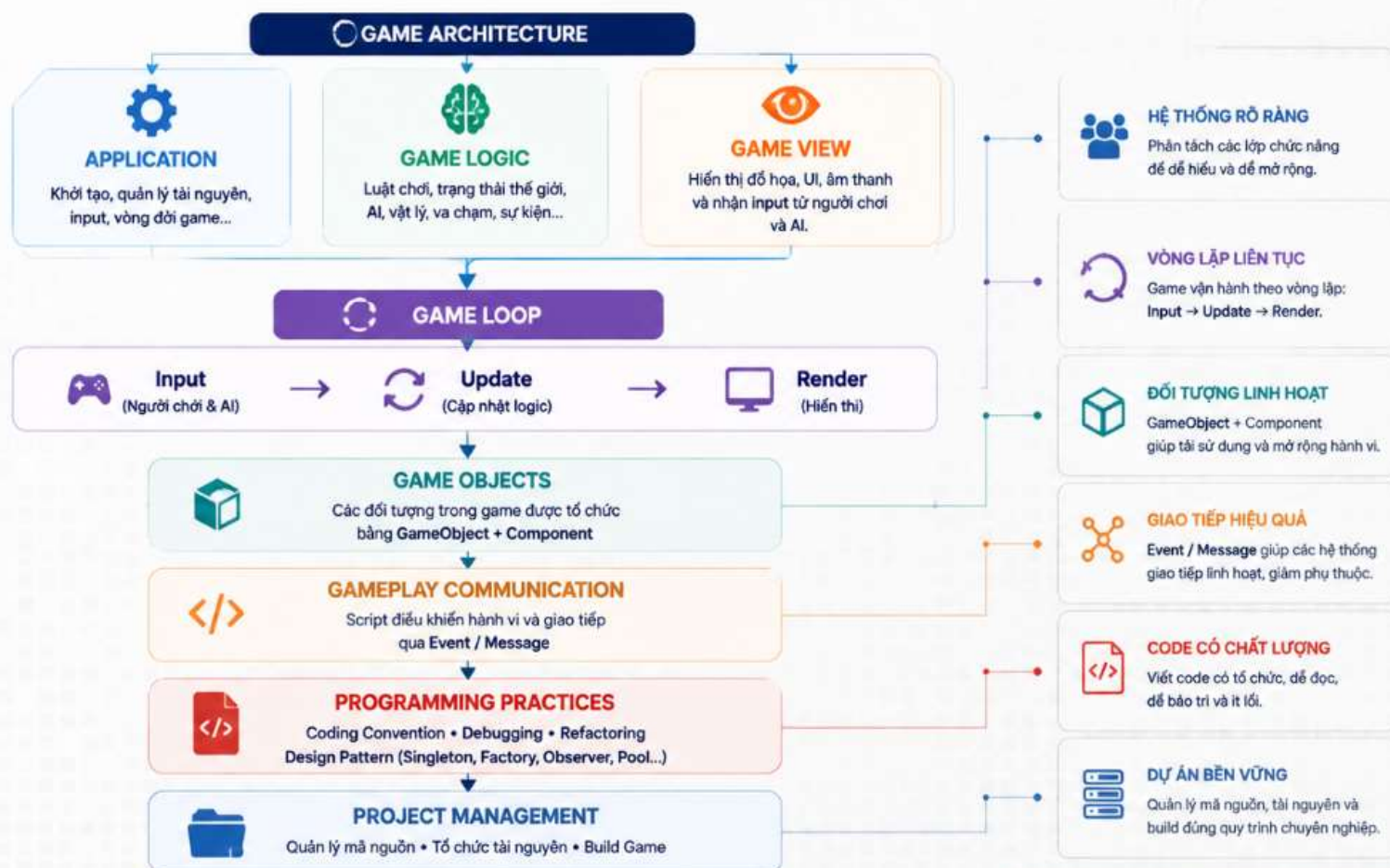
Nắm được 3 lớp kiến trúc chính của game

Hiểu được vòng đời game qua Game Loop

Quản lý đối tượng trong game

Nắm được kỹ năng lập trình với các event

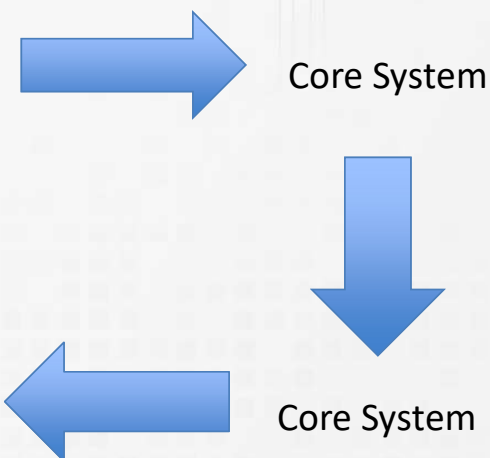
Hiểu được cách tổ chức dự án game



MỤC TIÊU

Hiểu kiến trúc tổng thể và vòng lặp của game, biết cách tổ chức đối tượng, giao tiếp hệ thống và áp dụng các kỹ thuật lập trình tốt để phát triển game bền vững.

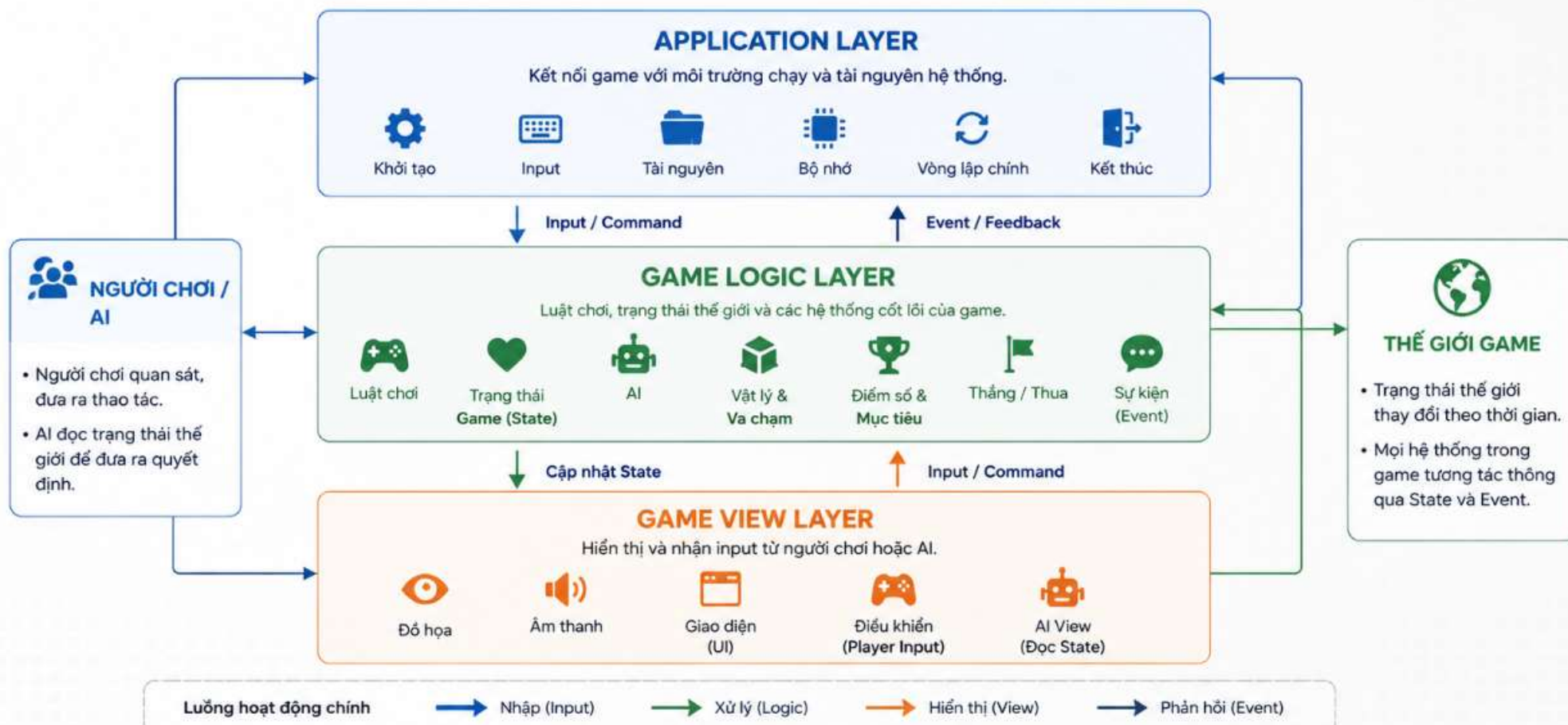
KIẾN TRÚC CỦA MỘT TRÒ CHƠI



Nếu tất cả gọi qua nhau thông qua core system thì project sẽ khó sửa, khó mở rộng, khó debug

Hệ thống game cần một kiến trúc cụ thể để tối ưu và mở rộng hệ thống khi cần thiết

KIẾN TRÚC TỔNG THỂ ?



Application Layer làm gì?

APPLICATION LAYER

Kết nối game với môi trường chạy và tài nguyên hệ thống.



Khởi tạo

Khởi động hệ thống,
cấu hình game.



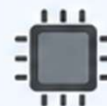
Input

Nhận input từ người
chơi (keyboard,
mouse, gamepad...)



Tài nguyên

Tải và quản lý
texture, model,
audio, data...



Bộ nhớ

Cấp phát và quản lý
bộ nhớ hiệu quả.



Vòng lặp chính

Điều khiển Game Loop:
Input → Update →
Render.



Kết thúc

Giải phóng tài nguyên,
lưu dữ liệu, tắt game
an toàn.

Application Layer trong Unity



UNITY ENGINE

Unity Engine đảm nhận hầu hết các công việc của **Application Layer**, để lập trình viên tập trung vào thiết kế gameplay và trải nghiệm.

- ✓ Quản lý hệ thống
- ✓ Quản lý tài nguyên
- ✓ Quản lý bộ nhớ
- ✓ Vòng lặp chính hiệu suất cao
- ✓ Quản lý vòng đời ứng dụng



Lập trình viên

Viết gameplay, logic, UI, hiệu ứng...

Nhấn Play



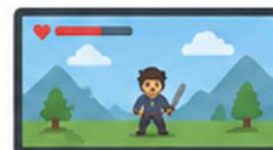
Unity khởi động

Khởi tạo engine và project.



Unity thực hiện Application Layer

Quản lý input, tài nguyên, bộ nhớ, vòng lặp, và kết thúc ứng dụng.



Game chạy

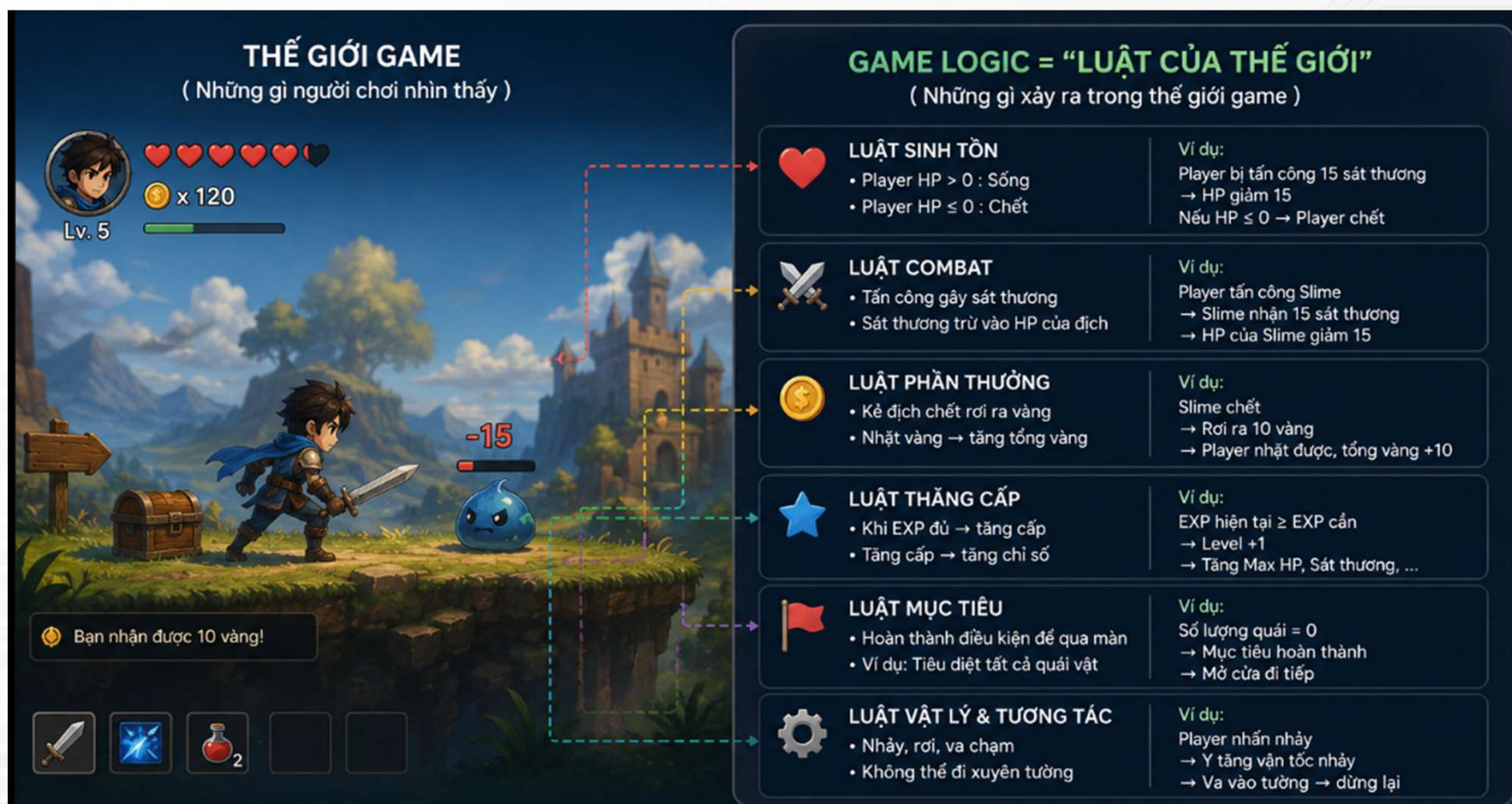
Lập trình viên chỉ cần chuyên tâm vào game logic và nội dung.



Lợi ích

- Tiết kiệm thời gian phát triển.
- Hiệu suất tối ưu.
- Ổn định và đáng tin cậy.
- Cho phép tập trung vào điểm khác biệt của game.

Game Logic Layer

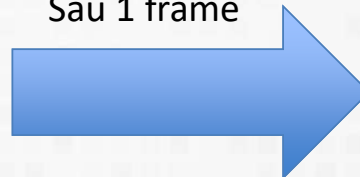


Game state

State (t)



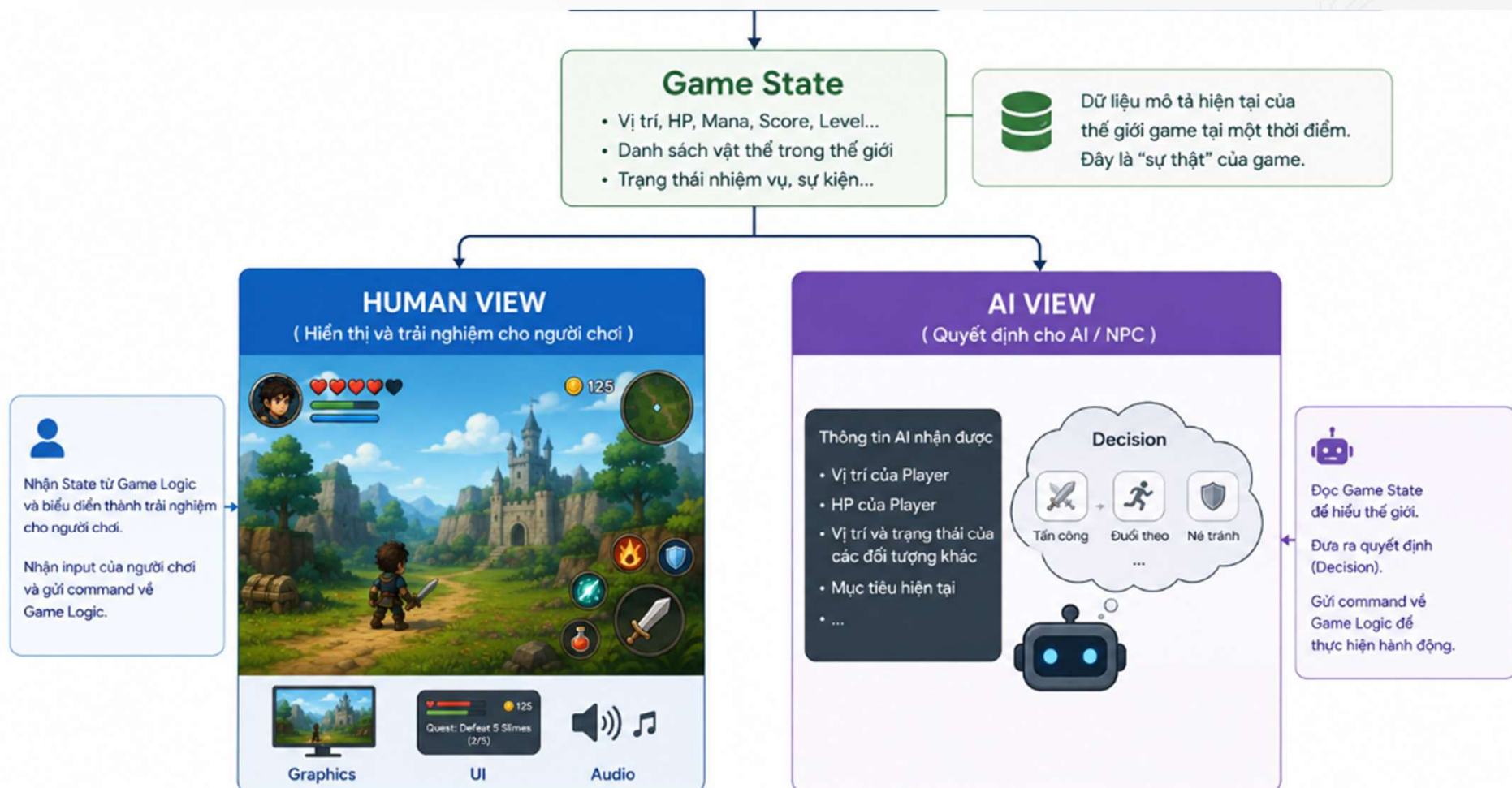
Sau 1 frame



State (t+1)



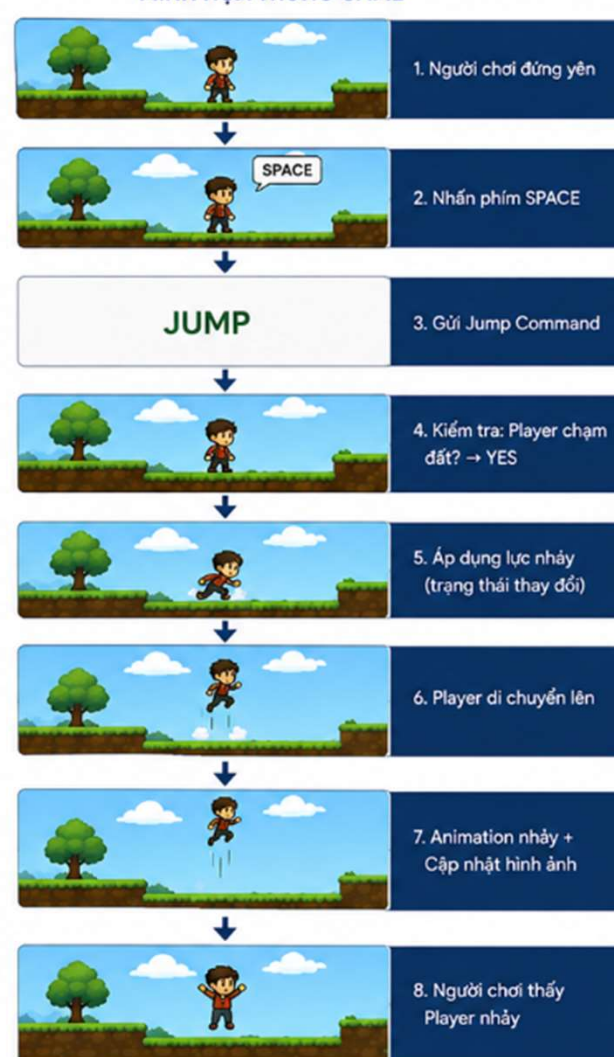
Game View Layer



LUỒNG XỬ LÝ (INPUT → OUTPUT)

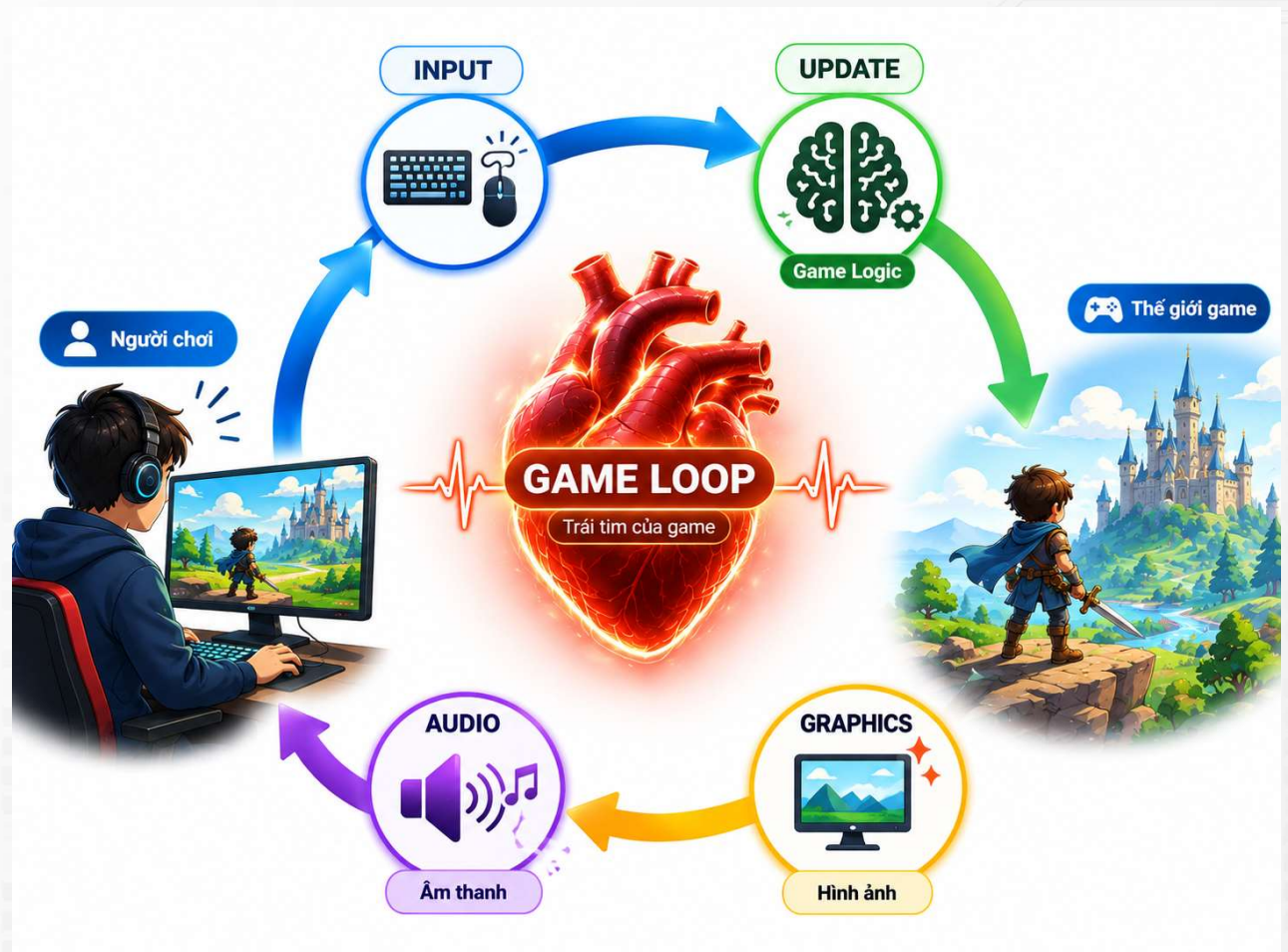


MINH HỌA TRONG GAME



VÒNG ĐỜI VÀ LƯỒNG THỰC THI

Game loop



Lưu ý Game loop trong Unity

Ví dụ với hàm Update ()

```
0 references
void Update()      unknown, 12 months ago • init ...
{
    if (_particleSystem.IsAlive())
    {
        UpdatePosition();
    }
}
```

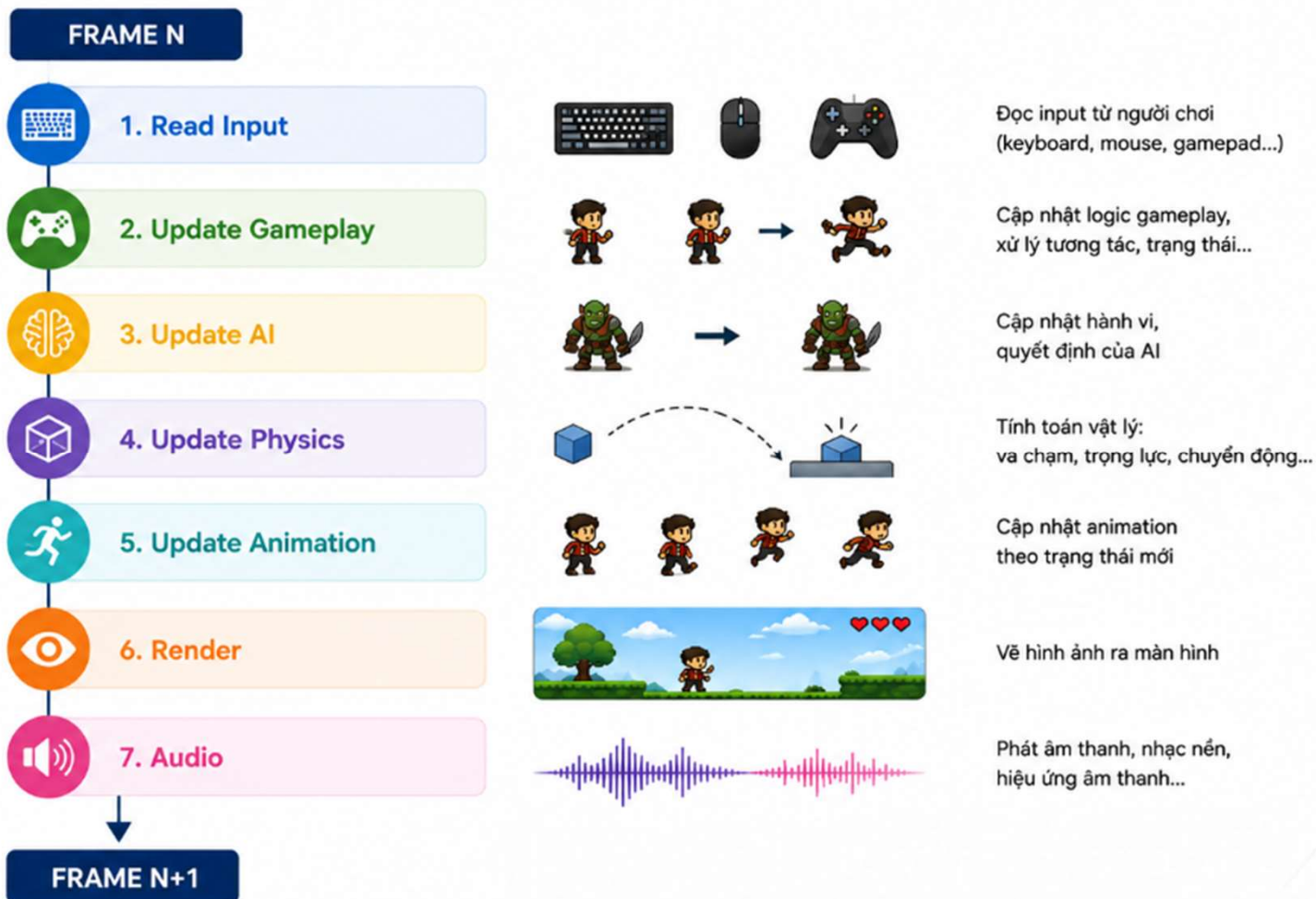
Tôi quyết định gọi hàm Update()



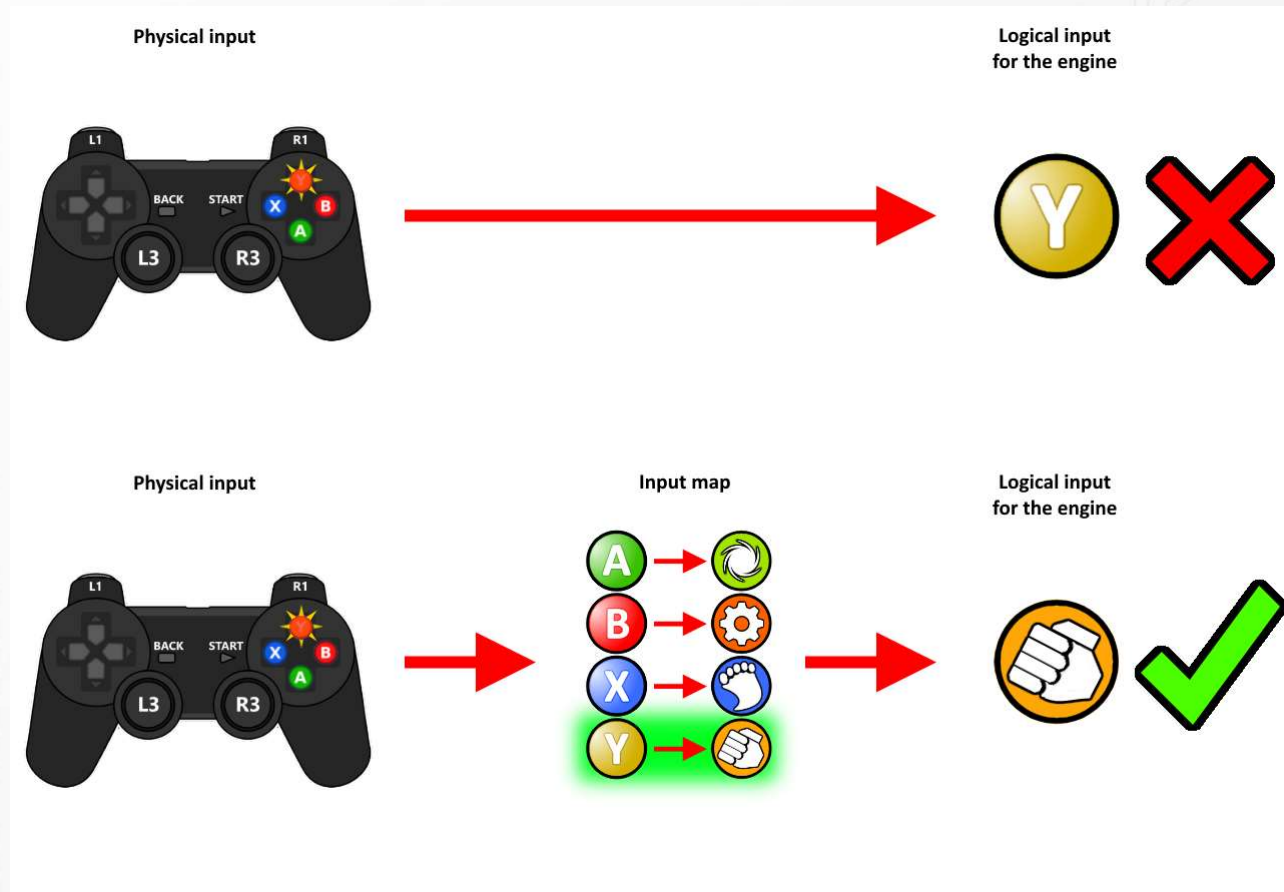
Chính GameLoop yêu cầu Unity gọi hàm Update()



Điều gì xảy ra trong 1 frame

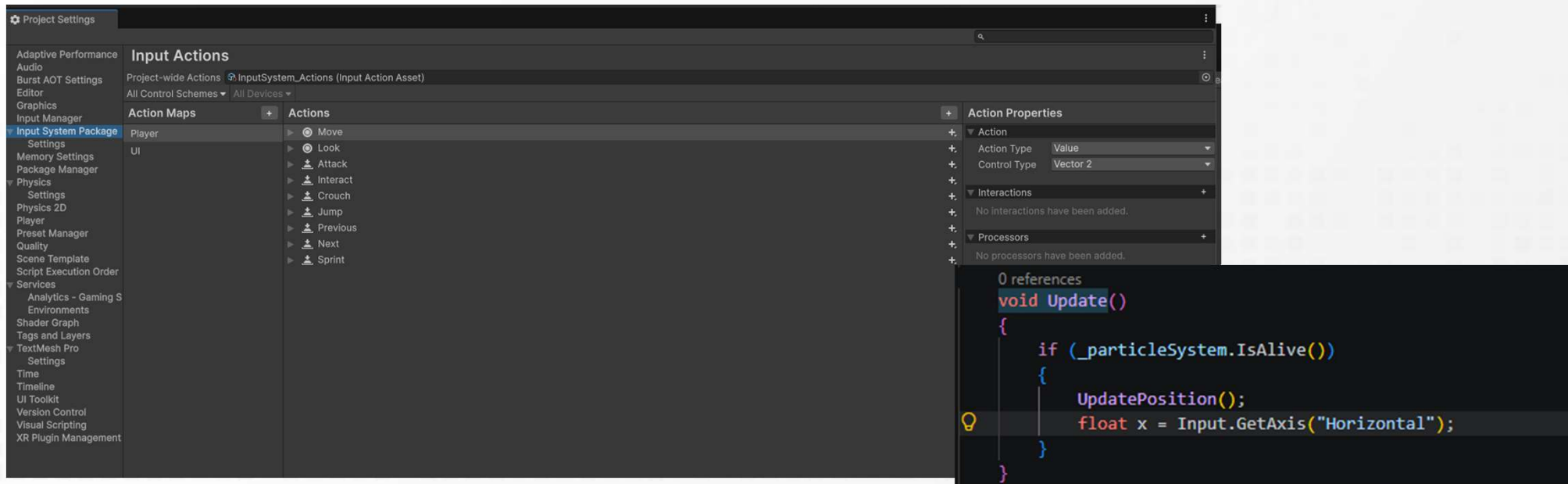


Input trong game



Input trong unity

Project Settings → Input / Input System



The screenshot displays the Unity Project Settings window, specifically the 'Input Actions' section. The left sidebar shows the 'Project Settings' hierarchy with 'Input System Package' expanded. The main area is divided into 'Action Maps' and 'Actions'. Under 'Action Maps', 'Player' is selected, showing a list of actions: Move, Look, Attack, Interact, Crouch, Jump, Previous, Next, and Sprint. The 'Actions' section on the right shows the 'Action Properties' for the selected action, including 'Action Type' (Value) and 'Control Type' (Vector 2). Below this, the 'Interactions' and 'Processors' sections are visible, both showing 'No interactions have been added' and 'No processors have been added' respectively. A code editor window is overlaid on the bottom right, showing a C# script snippet:

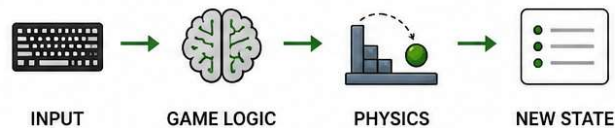
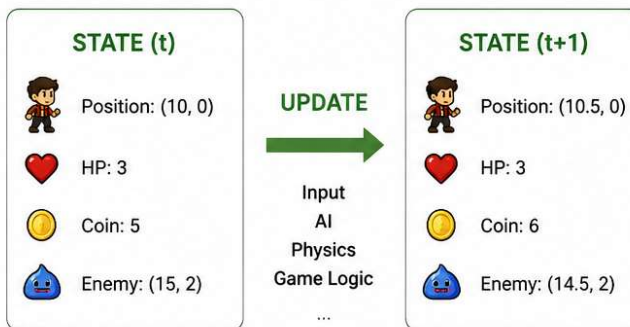
```
0 references
void Update()
{
    if (_particleSystem.IsAlive())
    {
        UpdatePosition();
        float x = Input.GetAxis("Horizontal");
    }
}
```

Update và Render



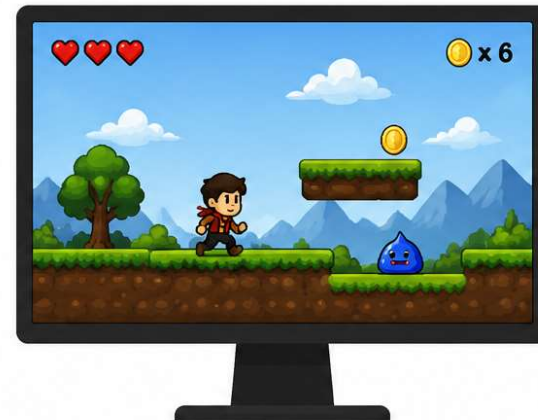
1. UPDATE

Cập nhật trạng thái thế giới game



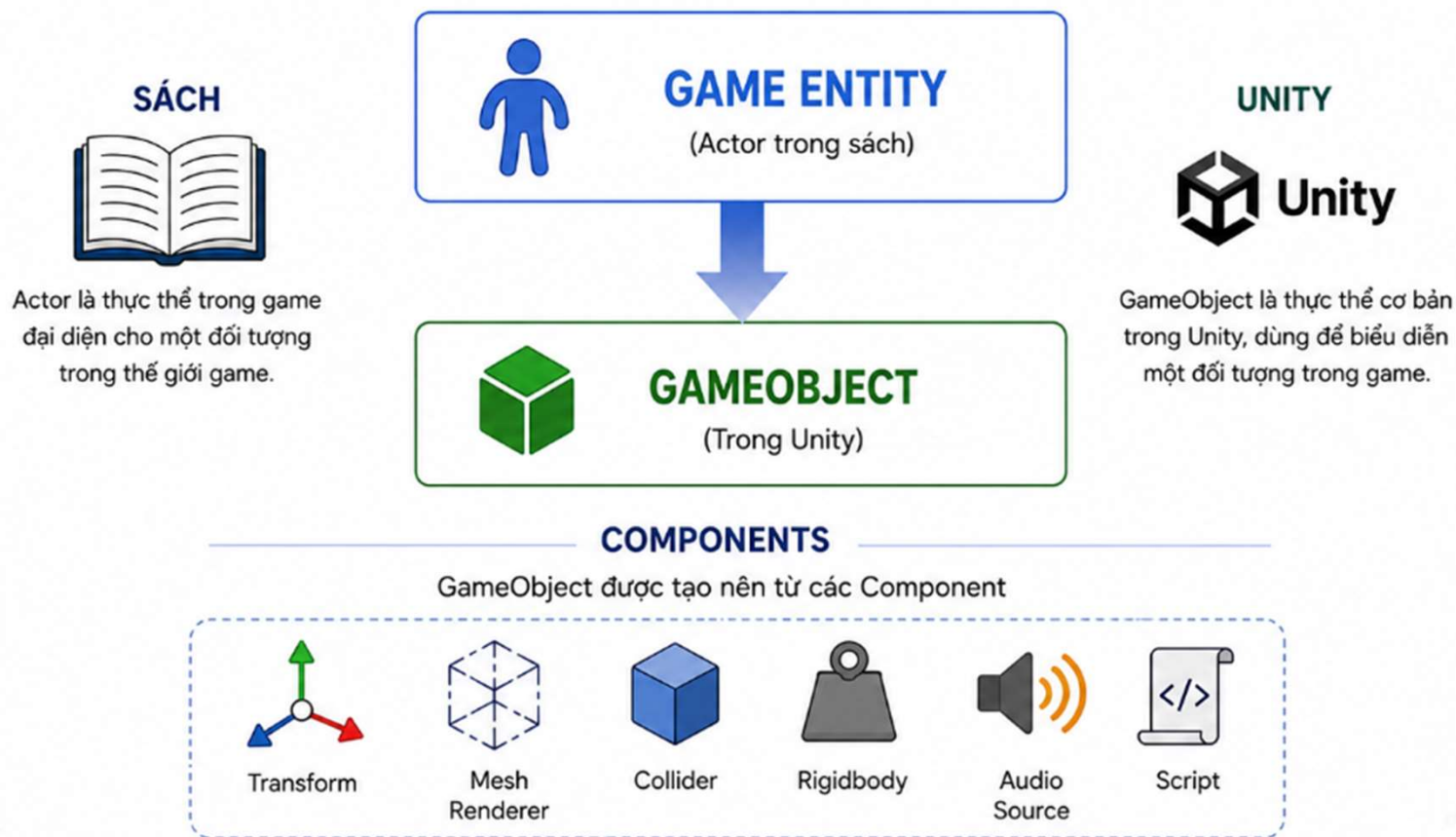
2. RENDER

Vẽ trạng thái mới ra màn hình



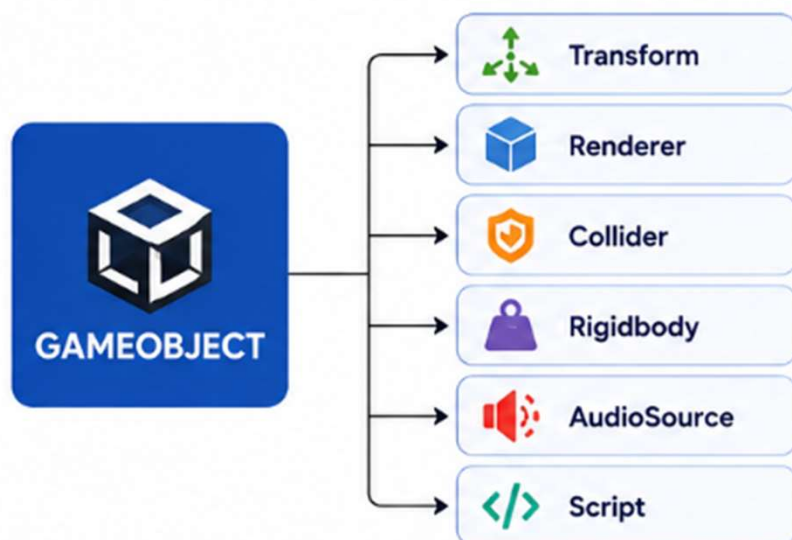
QUẢN LÝ ĐỐI TƯỢNG TRONG GAME

ĐỐI TƯỢNG TRONG UNITY



Nhắc lại : Game object là vật chứa

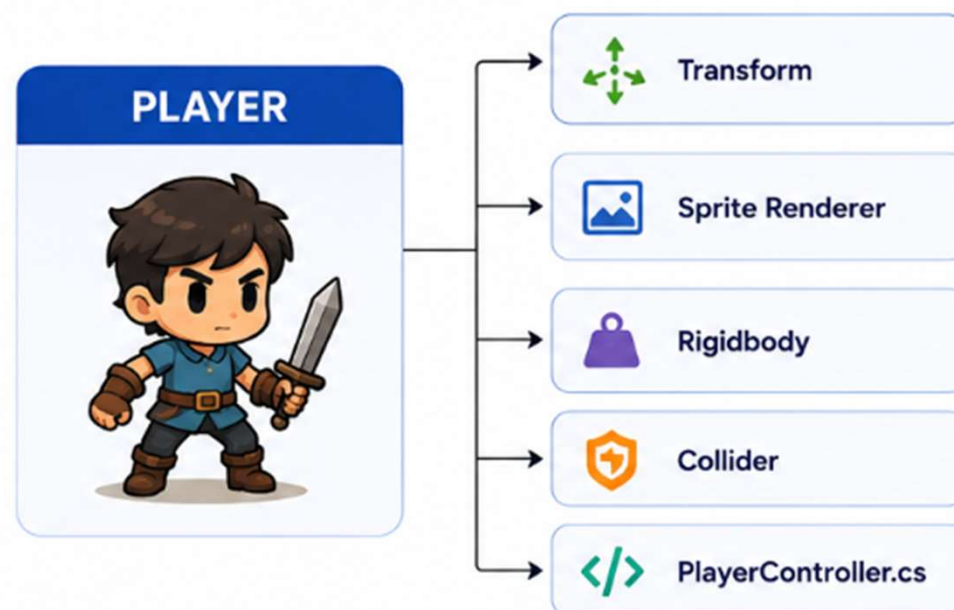
- Một trong những khái niệm đầu tiên:



 GameObject là “**vật chứa**”.

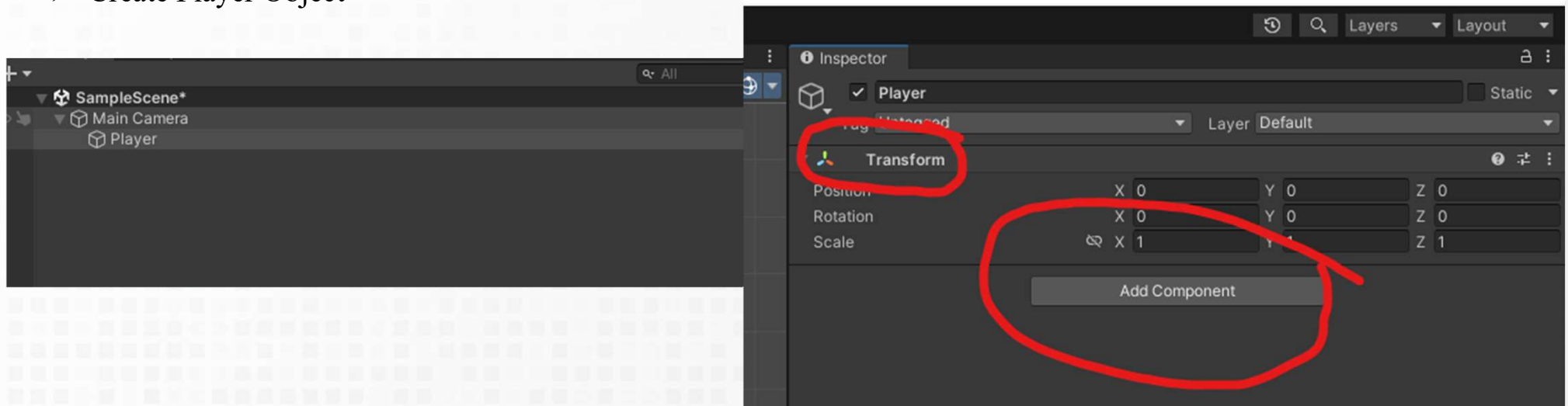
 Component cung cấp **khả năng**.

- Ví dụ:



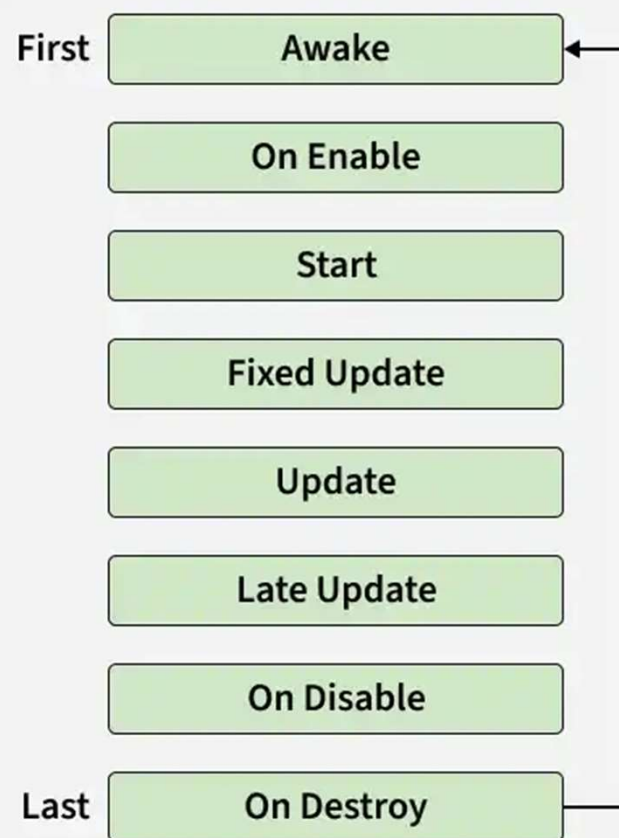
Tạo game Object trên Unity

- Hierarchy (right click)
- Create empty
- Create Player Object

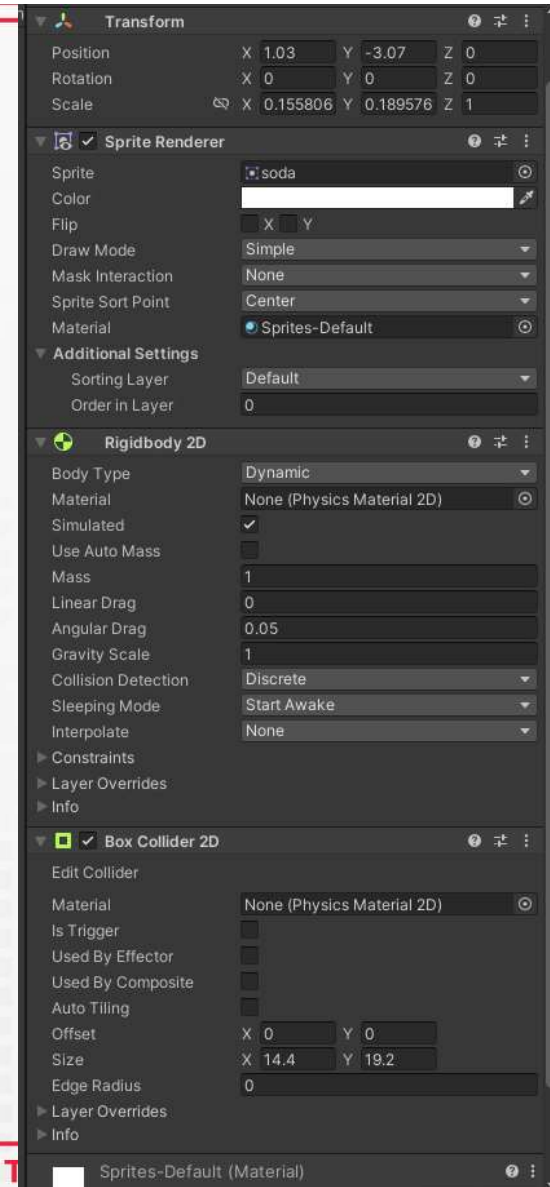
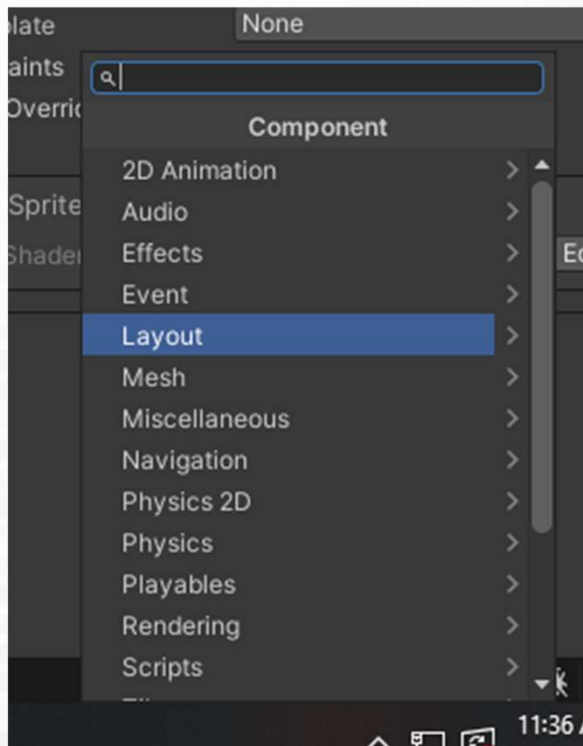


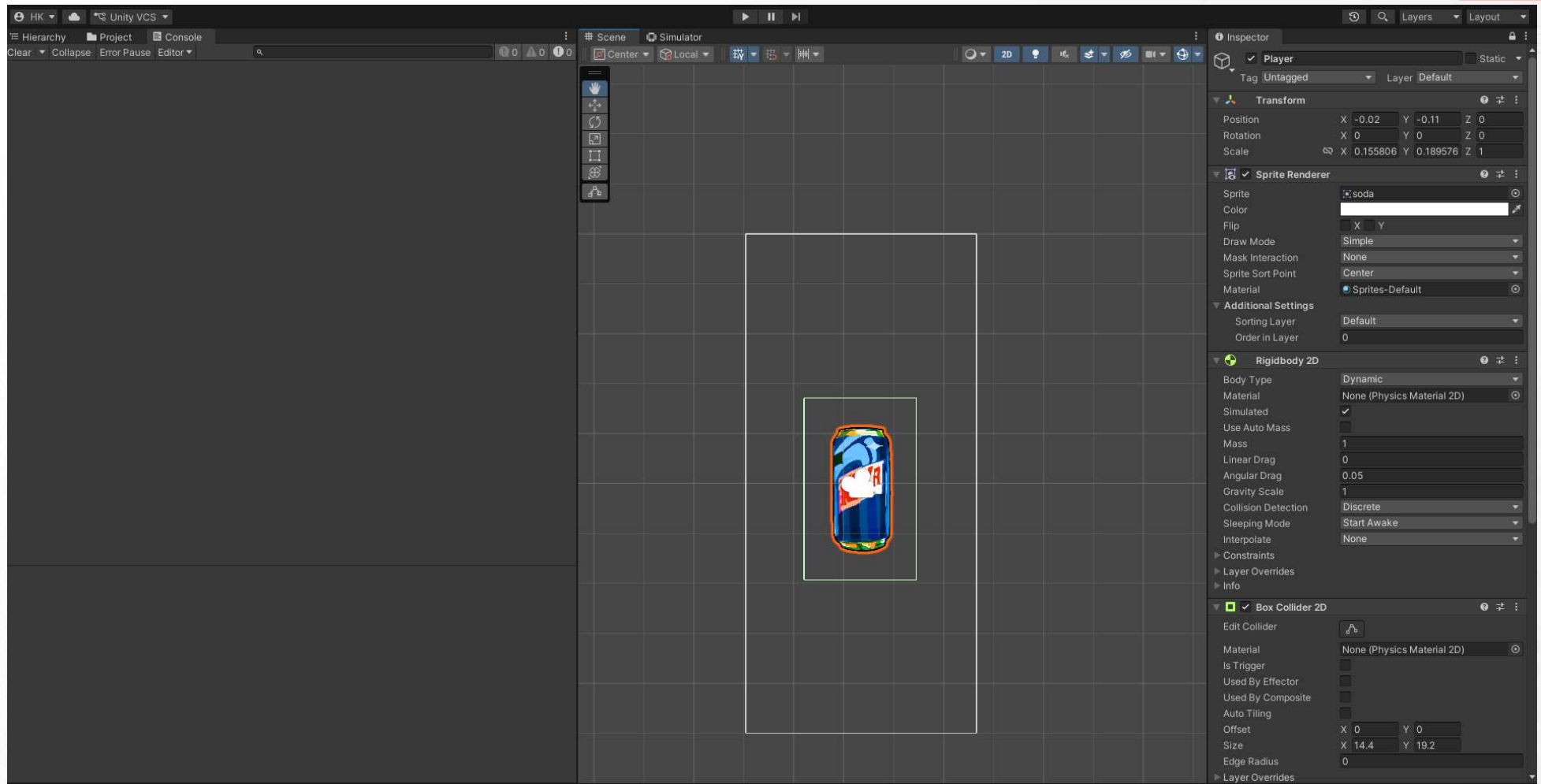
Component Architecture

Lớp MonoBehaviour trong C#, trong lập trình Unity, hầu hết mọi Script C# bạn tạo ra đều mặc định kế thừa từ lớp MonoBehaviour. Đây chính là lớp định nghĩa "hành vi" cho các đối tượng (GameObjects) trong thế giới game

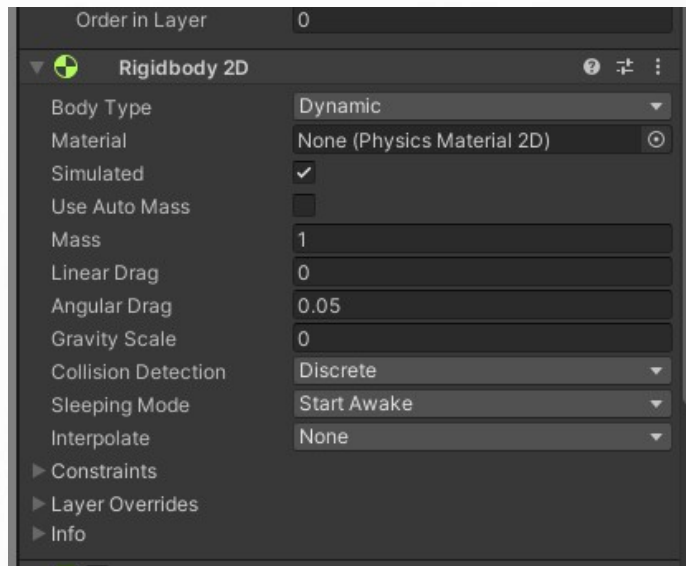


Thực hành với Component





Một vài chú ý



Hệ số trọng lực, nếu Gravity Scale >0 , object tự động rơi xuống đất (có thể thử)

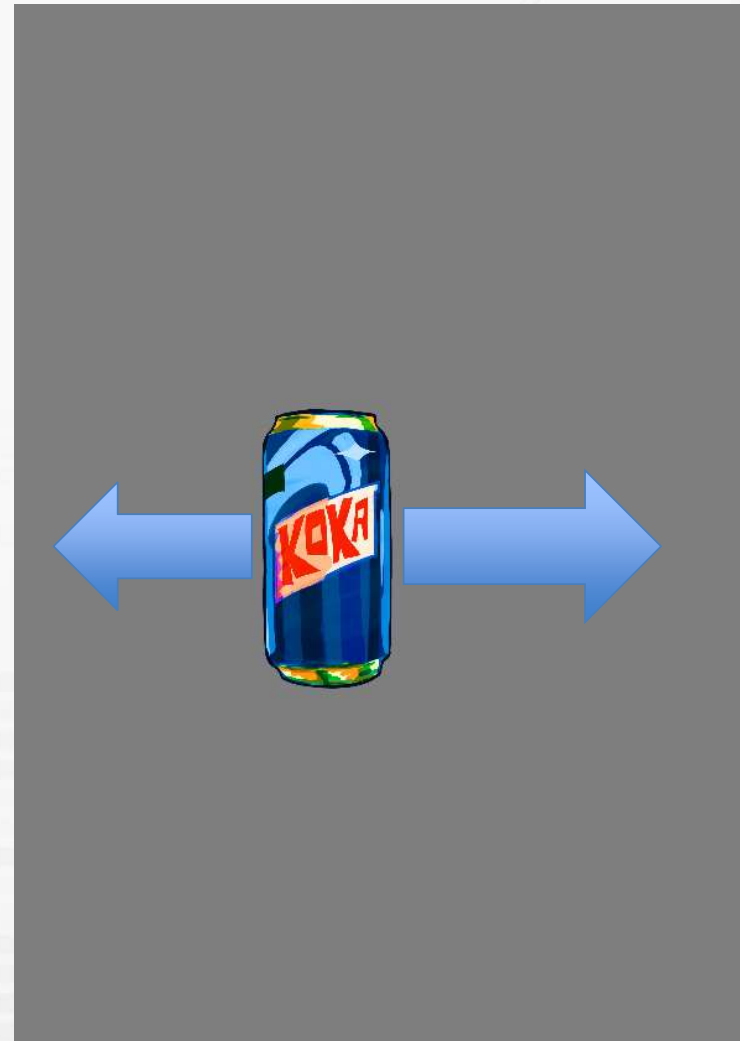
```

0 references
public class PlayerController : MonoBehaviour
{
    // Start is called before the first frame update
    1 reference
    public float moveSpeed = 5f;

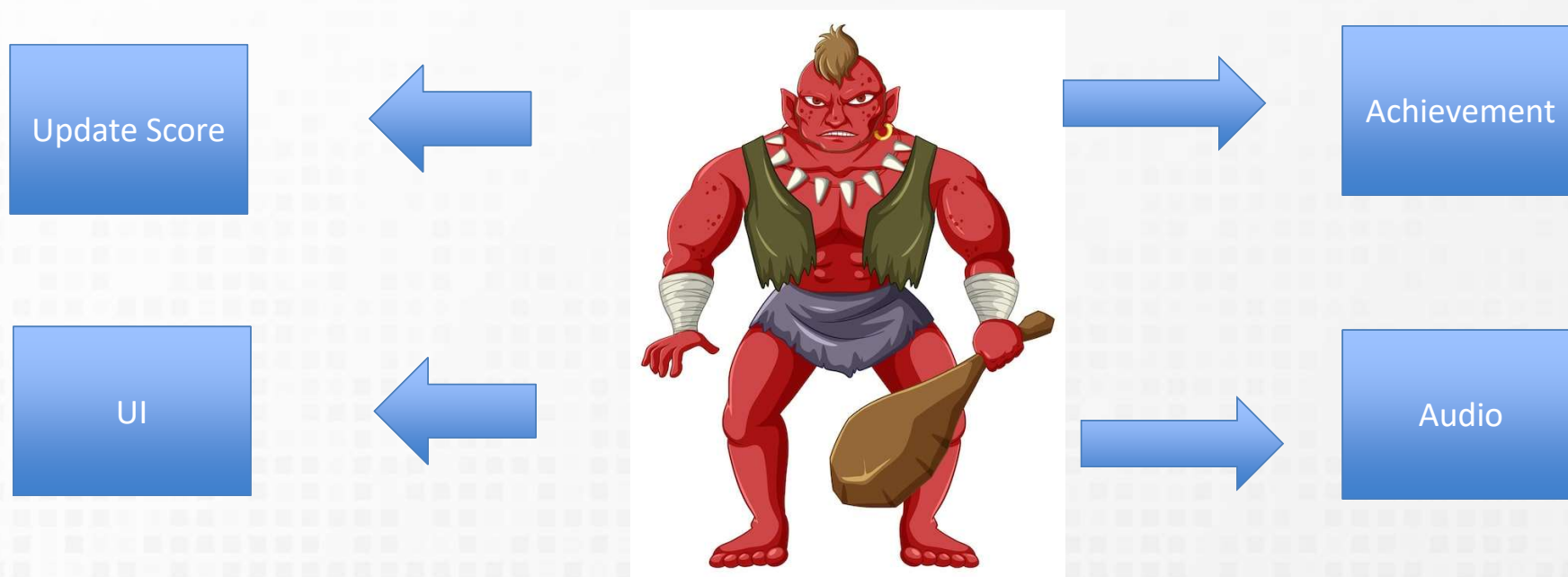
    3 references
    private Rigidbody2D rb;

    0 references
    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    0 references
    void Update()
    {
        float x = Input.GetAxisRaw("Horizontal");
        ⚡ rb.velocity = new Vector2(x * moveSpeed, rb.velocity.y);
    }
}
    
```



Vấn đề khi xử lý Object



Event là gì

VÍ DỤ SỰ KIỆN

-  Người chơi nhấn phím A
-  Chuột được nhấp
-  Nhân vật chạm vào vật phẩm
-  Nhân vật bị mất máu
-  Hết thời gian
-  Hoàn thành màn chơi

CƠ CHẾ HOẠT ĐỘNG CỦA EVENT

1. SỰ KIỆN XẢY RA



Người chơi hoặc hệ thống kích hoạt sự kiện

2. HỆ THỐNG LẮNG NGHE



Hệ thống lắng nghe sự kiện thông qua Event Listener

3. PHẢN HỒI (HANDLER)



Hàm xử lý (Handler) được gọi và thực thi phản ứng tương ứng



Ví von:

Event giống như "tiếng chuông cửa". Khi có người nhấn chuông (sự kiện), bạn nghe thấy (lắng nghe) và ra mở cửa (xử lý).

VÍ DỤ CODE (UNITY C#)

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    void Update()
    {
        // Lắng nghe sự kiện: nhấn phím Space
        if (Input.GetKeyDown(KeyCode.Space))
        {
            Jump(); // Xử lý sự kiện
        }
    }

    void Jump()
    {
        Debug.Log("Player Jump!");
        // Thực hiện nhảy
    }
}
```

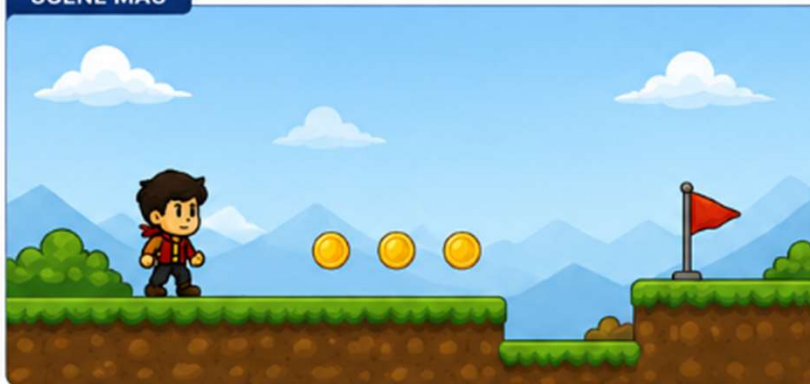
Sự kiện ở đây là: người chơi nhấn phím Space. Khi sự kiện xảy ra, hàm `Jump()` sẽ được gọi.

Bài tập hướng dẫn sử dụng object và component

MỤC TIÊU

- ✓ Tạo các GameObject và gắn đúng Component.
- ✓ Điều khiển Player di chuyển và nhảy.
- ✓ Phát hiện va chạm (Trigger) và xử lý.
- ✓ Thu thập Coin, tăng Score.
- ✓ Đến Goal để hoàn thành màn chơi.

SCENE MẪU



NHIỆM VỤ

- 1 Player di chuyển trái/phải bằng A/D hoặc ←/→.
- 2 Player nhảy bằng Space.
- 3 Player không rơi xuyên Ground.
- 4 Khi Player chạm Coin: Coin biến mất, **Score +1**.
- 5 Khi Player chạm Goal: `Debug.Log("LEVEL COMPLETE")`

Tổng kết

01 KIẾN TRÚC TỔNG THỂ



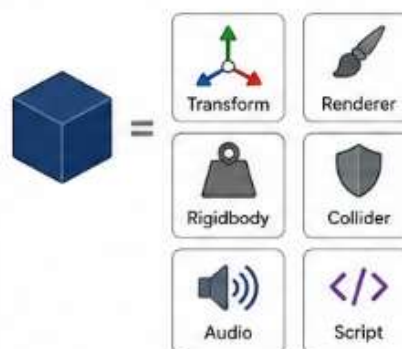
Ba lớp chính phối hợp để tạo nên một trò chơi hoàn chỉnh.

02 VÒNG ĐỜI GAME (GAME LOOP)



Lặp liên tục để game phản hồi và hiển thị mượt mà.

03 GAMEOBJECT & COMPONENT



GameObject là đối tượng trong game. Component bổ sung khả năng và hành vi.

04 EVENT



Event giúp tách biệt và kết nối các hệ thống một cách linh hoạt.

NHỮNG GÌ ĐÃ HỌC



Hiểu kiến trúc 3 lớp của game



Nắm được cơ chế Game Loop



Làm việc với GameObject và Component



Xử lý hành vi bằng Script



Hiểu và sử dụng Event trong game

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG CDIT